API Apartamentos

La API Apartamentos es una aplicación web para administrar la renta de casas, pisos y habitaciones. Permite gestionar propiedades, arrendatarios, propietarios, disponibilidad, reservas, pagos y reportes. Está desarrollada con Java 21, Spring Boot, Spring Data JPA, Thymeleaf y MySQL.

Repositorio de GitHub

https://github.com/mafercova/api2_apartamentos

Código fuente del proyecto en GitHub

API de propiedades

Registra y administra casas y pisos con su propietario, tipo, título, descripción, dirección, ubicación, precio por noche, capacidad, habitaciones, baños, superficie, comodidades, reglas y estado. Permite registrar, listar, consultar, actualizar y eliminar propiedades. Ruta: /api/propiedades.

API de unidades rentables

Administra habitaciones o espacios que pueden reservarse dentro de una propiedad. Cada unidad tiene nombre, tipo, capacidad, precio por noche y estado. Permite consultar unidades por propiedad. Ruta: /api/unidades.

API de clientes

Administra la información de arrendatarios y propietarios: nombres, apellidos, correo, teléfono, fecha de nacimiento, identificación, dirección y estado. El tipo CLIENTE representa al arrendatario y PROPIETARIO al propietario. Permite consultar el estado de cuenta de cada cliente. Ruta: /api/clientes.

API de reservas

Gestiona estancias relacionando un cliente y una unidad con fechas de entrada y salida. Valida fechas, disponibilidad y reservas solapadas. Calcula el importe total cuando no se envía y permite registrar, consultar, actualizar y eliminar reservas. Ruta: /api/reservas.

API de disponibilidad

Gestiona intervalos disponibles o bloqueados por unidad. Registra fecha inicial, fecha final, disponibilidad y precio especial opcional. Los intervalos bloqueados impiden registrar reservas coincidentes. Ruta: /api/disponibilidades.

API de pagos

Registra pagos asociados a una reserva. El cliente se obtiene de la reserva y se genera una referencia. Administra importe, método, fecha de vencimiento, fecha de pago y estado. Incluye operaciones de confirmación y reembolso; controla que los importes aplicados no superen el total de la reserva. Ruta: /api/pagos.

API de reportes y estados de cuenta

Genera reportes de ocupación, ingresos y morosidad, y calcula cargos, pagos confirmados, saldo y deuda vencida por cliente. Los reportes de ocupación e ingresos reciben las fechas desde y hasta; el inicio se incluye y el fin se excluye. Rutas: /api/reportes y /api/clientes/{id}/estado-cuenta.

API de imágenes de propiedades

Relaciona imágenes con propiedades mediante su URL y descripción. Permite registrar, consultar, actualizar, eliminar y listar imágenes por propiedad. Ruta: /api/propiedad-imagenes.

API de reseñas

Administra reseñas relacionadas con reservas y clientes. Registra calificación general, calificación de comunicación y comentario. Incluye consultas por reserva. Ruta: /api/resenas.

API de notificaciones

Administra mensajes entre remitente y destinatario con asunto, contenido, estado y fecha de envío. Permite consultar las notificaciones de un destinatario. Ruta: /api/notificaciones.

Interfaz web

- / - Resumen de operación.
- /propiedades y /unidades - Inventario de espacios.
- /clientes - Arrendatarios y propietarios.
- /reservas - Gestión de estancias.
- /pagos - Seguimiento de cobros.
- /reportes - Ocupación, ingresos y estados de cuenta.

Instrucciones para ejecutar localmente

Requisitos previos

- Java Development Kit 21.
- MySQL 8 y una base de datos llamada apartamentos.
- Git, para clonar el repositorio.
- Maven Wrapper incluido en el código.
- Postman, para realizar las pruebas de la API.

Clonar el repositorio

```
git clone https://github.com/mafercova/api2_apartamentos.git
cd api2_apartamentos
```

Crear la base de datos

En MySQL, ejecutar:

```
CREATE DATABASE IF NOT EXISTS apartamentos
CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

Configurar las variables de entorno

En PowerShell, definir las credenciales de MySQL y el puerto:

```
$env:DB_URL='jdbc:mysql://localhost:3306/apartamentos'
$env:DB_USERNAME='tu_usuario'
$env:DB_PASSWORD='tu_clave_mysql'
$env:SERVER_PORT='8000'
```

JAVA_HOME debe apuntar al JDK 21. El proyecto utiliza ddl-auto=update de forma predeterminada, por lo que las tablas se crean o actualizan al iniciar. La primera compilación requiere conexión a internet para descargar dependencias.

Ejecutar la aplicación

```
.\mvnw.cmd spring-boot:run
```

Abrir http://localhost:8000/. Para una base nueva, iniciar una vez la aplicación antes de ejecutar las pruebas completas, de modo que se creen las tablas.

Compilar y ejecutar las pruebas

Con MySQL y el esquema disponibles, detener la aplicación y ejecutar:

```
.\mvnw.cmd clean verify
java -jar target/apartamentos-0.0.1-SNAPSHOT.jar
```

También se puede ejecutar el JAR incluido en la entrega sin recompilar. Deben estar configuradas las credenciales y la base de datos del equipo.

Endpoints principales

URL base: http://localhost:8000. Las nueve APIs de entidades permiten consultar todos los registros mediante GET y crear mediante POST en su ruta base. GET, PUT y DELETE con /{id} permiten consultar, actualizar y eliminar un registro.

```
/api/clientes
/api/propiedades
/api/unidades
/api/reservas
/api/disponibilidades
/api/pagos
/api/propiedad-imagenes
/api/resenas
/api/notificaciones
```

Consultas y operaciones específicas

```
GET /api/unidades/propiedad/{idPropiedad}
GET /api/disponibilidades/unidad/{idUnidad}
GET /api/propiedad-imagenes/propiedad/{idPropiedad}
GET /api/resenas/reserva/{idReservacion}
GET /api/notificaciones/destinatario/{idCliente}
GET /api/pagos/reserva/{idReserva}
POST /api/pagos/{id}/confirmar
POST /api/pagos/{id}/reembolsar
GET /api/clientes/{id}/estado-cuenta
GET /api/reportes/ocupacion?desde=2026-10-01&hasta=2026-11-01
GET /api/reportes/ingresos?desde=2026-10-01&hasta=2026-11-01
GET /api/reportes/morosidad
GET /actuator/health
```

Pruebas de Apartamentos

La revisión del proyecto registró 28 pruebas automatizadas aprobadas, sin fallos, errores u omisiones. Incluyen el arranque con MySQL, el renderizado de pantallas, las reglas de servicios y el flujo de reserva y pago con persistencia real dentro de una transacción revertida.

- Relación propietario - propiedad - unidad.
- Relación cliente - reserva - pago.
- Rechazo de reservas en intervalos bloqueados.
- Validación de reservas solapadas.
- Confirmación y reembolso de pagos.
- Actualización del saldo en el estado de cuenta.
- Reportes de ocupación, ingresos y morosidad.

Peticiones para Postman

La entrega incluye Apartamentos.postman_collection.json. Importar el archivo en Postman y establecer baseUrl como http://localhost:8000. Configurar clienteId, unidadId, reservaId y pagoId con identificadores existentes antes de enviar solicitudes que modifican registros.

GET http://localhost:8000/api/clientes
GET http://localhost:8000/api/propiedades
GET http://localhost:8000/api/unidades
GET http://localhost:8000/api/reservas
GET http://localhost:8000/api/pagos
GET http://localhost:8000/api/disponibilidades
GET http://localhost:8000/api/reportes/morosidad

Orden recomendado para las pruebas

- Registrar un propietario y un arrendatario.
- Registrar la propiedad y una unidad rentable.
- Crear una reserva CONFIRMADA para el arrendatario.
- Registrar un pago parcial y confirmarlo.
- Consultar el estado de cuenta y los reportes.
- Intentar una reserva solapada y comprobar su rechazo.